

Table of Contents

Unit 1: Introduction to Computational Systems	4
Student Learning Outcomes	4
Introduction	5
1.1 Theory of Systems	5
What Is a System?	5
The Hook	5
The Four Basic Concepts of a System	6
1. Objective	6
2. Components	6
3. Environment	6
4. Communication	7
Summary Table: The Four Concepts of a System	7
Class Activity: Design a Simple System	7
Interactive Stop-Point: Pause & Think	8
Quick Recap	8
1.2 Software	8
What Is Software?	8
The Hook	9
Types of Software	9
Type 1: System Software	9
Type 2: Application Software	9
Comparing System Software vs. Application Software	10
Class Activity	10
Interactive Stop-Point: Pause & Think	11
Quick Recap	11
1.3 The Von Neumann Architecture	11
The Blueprint of Every Modern Computer	11
The Hook	12
Components of the Von Neumann Architecture	12
Component 1: Memory	12
Component 2: Central Processing Unit (CPU)	12
Component 3: Input Devices	13
Component 4: Output Devices	13
Component 5: The System Bus	14
The Von Neumann Architecture — Component Map	15

Working of the Von Neumann Architecture: The Fetch-Decode-Execute-Store Cycle	15
The Hook.....	15
Step 1: Fetch.....	16
Step 2: Decode	16
Step 3: Execute.....	16
Step 4: Store	16
The Full Cycle — Example: Adding 2 + 2 in a Calculator App	17
Interactive Stop-Point: Grab a Partner	17
Quick Recap.....	17
1.4 Number Systems	17
Why Do Computers Use Different Number Systems?	17
The Hook.....	18
1.4.1 Decimal Number System (Base 10)	18
1.4.2 Binary Number System (Base 2)	18
Converting Decimal to Binary.....	19
Interactive Stop-Point: Pause & Think	20
1.4.3 Octal Number System (Base 8).....	20
Converting Binary to Octal	20
Converting Decimal to Octal	21
Interactive Stop-Point: Grab a Partner	22
1.4.4 Hexadecimal Number System (Base 16)	22
Converting Hexadecimal to Binary.....	23
Converting Decimal to Hexadecimal	24
Interactive Stop-Point: Pause & Think	24
Quick Recap.....	24
1.5 Data Representation in Computing Systems	25
How Computers Store Numbers.....	25
The Hook.....	25
1.5.1 Whole Numbers (W)	25
1.5.2 Integers (Z) — Signed Numbers.....	25
1.5.3 1's Complement.....	26
1.5.4 2's Complement — How Computers Store Negative Numbers	26
Example: Represent -5 in 8-bit binary using 2's complement	27
Minimum Integer Value	27
Interactive Stop-Point: Pause & Think	28
Quick Recap.....	28
1.6 Binary Arithmetic Operations	28
Introduction.....	28

The Hook.....	28
1.6.1 Binary Addition.....	28
Example: Add $1101_2 + 1011_2$	29
1.6.2 Binary Subtraction (Using 2's Complement).....	29
Example: Subtract 6 from 9 (i.e., $9 - 6$).....	30
1.6.3 Binary Multiplication.....	30
Example: Multiply $101_2 \times 11_2$	31
1.6.4 Binary Division.....	31
Example: Divide $1100_2 \div 10_2$	31
Class Activity: Practicing Binary Division.....	32
Interactive Stop-Point: Grab a Partner.....	32
Quick Recap.....	32
1.7 Common Text Encoding Schemes.....	33
How Computers Represent Text.....	33
The Hook.....	33
1.7.1 ASCII.....	33
Class Activity.....	34
1.7.2 Unicode.....	35
UTF-8.....	35
UTF-16.....	36
UTF-32.....	36
Comparison: ASCII vs. Unicode Encodings.....	37
1.7.3 Data Storage Units.....	37
Interactive Stop-Point: Pause & Think.....	38
Quick Recap.....	38
Chapter Summary.....	38
Key Vocabulary.....	40
Review Questions.....	42
Multiple Choice Questions.....	42
Short Questions.....	43
Long Questions.....	44
Practical Exercises.....	44

Unit 1: Introduction to Computational Systems

"Computers are incredibly fast, accurate, and stupid. Human beings are incredibly slow, inaccurate, and brilliant. Together they are powerful beyond imagination."

— Albert Einstein

Student Learning Outcomes

By the end of this chapter, you will be able to:

- Define and describe **general system theory** — its types, objectives, components, and environment.
 - Explain the importance of **system objectives** such as processing information, supporting applications, and achieving specific goals.
 - Describe a **computer as a system**, including its objectives, architecture, components, and interactions.
 - Recognize the role and importance of **computer system components** and how they interact.
 - Understand the **Von Neumann architecture** and its core components: CPU, memory, input/output devices, and the system bus.
 - Explain the relationship between the **CPU, memory, and storage**, and how data flows within a system.
 - Describe how the CPU **fetches, decodes, executes, and stores** instructions.
 - Understand **number systems** — decimal, binary, octal, and hexadecimal — and convert between them.
 - Understand how **whole numbers and integers** are represented in binary, including signed and unsigned numbers.
 - Perform **binary arithmetic operations**: addition, subtraction, multiplication, and division.
 - Explain **1's complement** and **2's complement** and how they represent negative numbers.
 - Understand **text encoding schemes** — ASCII and Unicode — and how computers represent characters.
 - Differentiate between **system software and application software** and explain their roles.
-

Introduction

Every day, you interact with dozens of systems without realizing it. The traffic lights on the road. The WhatsApp message that reaches your friend in seconds. The refrigerator that keeps your food cold. Your own body, breathing and pumping blood without you thinking about it.

All of these are **systems** — organized collections of parts working together toward a goal.

A computer is also a system. But it is a special kind of system — one that can be **programmed** to do almost anything: calculate the trajectory of a rocket, play a game, translate languages, or draw a picture.

In this chapter, you will learn what a system is, how a computer is built, how it stores and processes information, and how it represents everything — from the letter "A" to a photo — using just two digits: **0** and **1**.

This is where computer science begins.

1.1 Theory of Systems

What Is a System?

Definition: A system is an organized collection of interdependent parts that work together to perform a specific function or achieve a common goal.

The key word is **interdependent** — the parts need each other. Remove one part and the whole system is affected.

The Hook

Think about your school. It has students, teachers, classrooms, timetables, a principal, a canteen, and textbooks. None of these parts alone makes a school. But when they all work together — students attend classes, teachers teach, the timetable organizes everything — the school achieves its goal: **education**.

Your school is a system.

Now think about a computer. It has a processor, memory, a keyboard, a screen, and software. None of these alone can do anything useful. But together, they form a **computational system** that can process, store, and display information.

The Four Basic Concepts of a System

Every system — simple or complex — can be described using four concepts:

1. Objective

Definition: The objective is the **purpose** or **goal** the system exists to achieve.

Understanding a system's objective is the first step to understanding how it works.

System	Objective
Transport system	Move people and goods safely between locations.
Computer system	Process data and provide useful information to users.
Human body	Sustain life and enable movement, thought, and sensation.
School	Educate students and prepare them for the future.

Without an objective, a system has no direction. Every decision about the system's design comes back to: **does this help achieve the goal?**

2. Components

Definition: Components are the **building blocks** of a system. Each component plays a specific role and contributes to the overall function.

Understanding the role of each component helps you:

- Identify problems when something goes wrong.
- Improve performance by upgrading key parts.
- Redesign the system intelligently.

Example: In a computer system, the components include the CPU, RAM, hard disk, keyboard, and monitor. Each has a specific job — and when they all work smoothly together, the system meets its objective.

3. Environment

Definition: The environment of a system is everything **external** to the system that interacts with it — inputs coming in, outputs going out, and outside factors affecting it.

The environment shapes how a system behaves.

Example: A computer's environment includes the user (who gives input), the electricity supply (which powers it), and temperature (overheating can cause failure). Intelligent systems sense their environment and adjust. A laptop's fan speeds up when it detects heat. A phone's screen dims in bright sunlight.

4. Communication

Definition: Communication is the way **components interact** with each other to achieve the system's objective.

Without communication between parts, a system falls apart.

Example (Computer): The CPU communicates with RAM to fetch and store data. The keyboard sends signals to the CPU when a key is pressed. The CPU sends results to the monitor for display.

Example (Biological): Your brain sends electrical signals to your hand muscles when you decide to pick up a pen. That signal is communication between components of the biological system.

Summary Table: The Four Concepts of a System

Concept	Question It Answers	Computer Example
Objective	What is the system trying to do?	Process data, provide information.
Components	What parts make up the system?	CPU, RAM, keyboard, monitor.
Environment	What surrounds and affects the system?	User input, electricity, temperature.
Communication	How do parts talk to each other?	CPU ↔ RAM, keyboard → CPU → monitor.

Class Activity: Design a Simple System

Work in pairs. Choose any simple system from daily life (a vending machine, a water pump, a traffic light, a kitchen blender).

For your chosen system:

1. **Objective:** What is it trying to do?
2. **Components:** List all the parts.
3. **Environment:** What external factors affect it?
4. **Communication:** How do the components interact?

Draw a diagram showing your system. Present it to the class.

Interactive Stop-Point: Pause & Think

Think about the **human digestive system** (the system that processes food).

1. What is its **objective**?
2. Name four **components** (organs).
3. What is the **environment** — what goes in? What comes out?
4. How do the components **communicate** (what signals or substances pass between them)?

Write your answers. Then compare with a partner.

Quick Recap

A system is a collection of interdependent parts with a shared goal. Every system has four concepts: **objective, components, environment, and communication.**

1.2 Software

What Is Software?

Definition: Software is a collection of programs and instructions that tell a computer what to do and how to do it.

Without software, a computer is a useless box of metal and plastic. Software is what brings hardware to life.

The Hook

In 1971, a programmer created a small program just to see if it could copy itself from one computer to another over a network. It could. The program called itself "**Creeper**" and displayed the message: *"I'm the creeper, catch me if you can!"*

It was the world's first computer virus — and it was created with software.

Software can be creative, useful, harmful, or playful. But it is always a set of instructions telling the hardware what to do.

Types of Software

Software is divided into two main categories:

Type 1: System Software

Definition: System software manages the computer's hardware resources and provides a platform on which application software can run. It acts as a bridge between the hardware and the user.

Think of system software as the **manager** of the computer. It runs quietly in the background, making sure everything works correctly.

Examples of system software:

Type	Examples	What It Does
Operating System (OS)	Windows, macOS, Linux, Android	Controls all hardware; manages files, memory, and programs.
Device Drivers	Printer driver, graphics driver, sound driver	Allows the OS to communicate with specific hardware devices.
Utility Programs	Antivirus, disk cleanup, backup software	Maintains and protects the system.

Type 2: Application Software

Definition: Application software is designed to help users perform specific tasks. It runs on top of the system software.

Think of application software as the **tools** you actually use to get work done.

Examples of application software:

Type	Examples
Word Processors	Microsoft Word, Google Docs
Web Browsers	Google Chrome, Mozilla Firefox, Safari
Games	Minecraft, Fortnite, Among Us
Media Players	VLC Media Player, Windows Media Player
Spreadsheets	Microsoft Excel, Google Sheets

Comparing System Software vs. Application Software

Feature	System Software	Application Software
Purpose	Manages hardware; makes the computer run.	Helps the user complete a specific task.
Examples	Windows OS, device drivers, antivirus	MS Word, Chrome, Minecraft
Installation	Usually pre-installed. Comes with the computer.	Installed by the user as needed.
Who uses it directly?	Mostly works in background; user rarely touches it.	User directly interacts with it.
Runs on	Directly on the hardware.	Runs on top of system software.

Important: Application software **cannot run without system software**. If you remove the operating system, no application — Word, Chrome, games — can run. System software comes first.

Class Activity

Make a list of every software program you use on your computer, tablet, or phone. Then sort each one into **system software** or **application software**.

Software Name	System or Application?	What Does It Do?
Windows 11	System	Manages all hardware and software
Microsoft Word	Application	Writing documents
...	...	...

Compare your list with a classmate. Which software do you use most?

Interactive Stop-Point: Pause & Think

Your friend says: *"I don't need an operating system. I'll just install Microsoft Word directly on the computer."*

Is your friend correct? Why or why not? What would happen if you tried to run Word without an operating system?

Quick Recap

Software tells hardware what to do. System software manages the computer (OS, drivers, utilities). Application software helps users complete specific tasks (Word, Chrome, games). One cannot work without the other.

1.3 The Von Neumann Architecture

The Blueprint of Every Modern Computer

Definition: The Von Neumann architecture is a computer design model in which a single memory stores both the program instructions and the data those instructions operate on. The CPU reads from this shared memory to execute programs.

This model was developed in the 1940s by mathematician **John von Neumann**. Before this model, computers were rewired physically every time you wanted to run a different program. Von Neumann's idea — store the program in memory, just like data — changed everything.

Almost every computer built since the 1950s is based on this architecture. Your laptop, your phone, your school's computer — all of them are Von Neumann machines.

The Hook

Imagine a chef (the CPU) in a kitchen. The chef reads a recipe (the program) from a cookbook (memory). The recipe tells the chef which ingredients (data) to use and what to do with them. The chef follows the steps, one by one, and produces a meal (output).

Now imagine if the chef had to memorize every possible recipe permanently. That would be impossible. Instead, the cookbook stores all the recipes. The chef reads whichever one is needed.

That is the Von Neumann idea: **store the instructions in memory, read them as needed, execute them one by one.**

Components of the Von Neumann Architecture

Component 1: Memory

Role: Stores both the **program instructions** and the **data** the program works with.

When you open a program (like a game or a word processor), it is **loaded from the hard disk into RAM** (Random Access Memory). The CPU then reads instructions and data from RAM — because RAM is much faster than the hard disk.

Think of RAM as the **chef's countertop** — the working space where everything currently being used is laid out.

Key point: Von Neumann architecture uses a **single, shared memory** for both instructions and data. This is called the **stored-program concept**.

Component 2: Central Processing Unit (CPU)

Role: The brain of the computer. It fetches instructions from memory, decodes them, and executes them.

The CPU has two main sub-units:

Sub-Unit	Full Name	What It Does
ALU	Arithmetic Logic Unit	Performs all mathematical calculations (addition, subtraction, etc.) and logical operations (AND, OR, NOT, comparisons).
CU	Control Unit	Controls the flow of instructions. Tells the ALU and memory what to do and when. Acts like a traffic controller for data inside the CPU.

Example: When you calculate $2 + 2$ on a calculator app:

- The **Control Unit** reads the instruction "add these two numbers."
- The **ALU** performs the actual addition.
- The **Control Unit** sends the result to the display.

Component 3: Input Devices

Role: Allow users to send data and instructions **into** the computer system.

Input Device	What It Sends
Keyboard	Text and commands
Mouse	Cursor position and click actions
Microphone	Sound/audio
Scanner	Images and documents
Camera (webcam)	Video/images

When you press a key on the keyboard, the keystroke is converted to a digital signal and sent to the CPU for processing.

Component 4: Output Devices

Role: Receive the results of the CPU's processing and present them to the user.

Output Device	What It Shows
Monitor	Visual output (text, images, video)
Printer	Physical printed documents
Speakers	Sound/audio
Projector	Large-screen visual output

After the CPU processes your data, the result travels to the monitor (or other output device) so you can see or hear it.

Component 5: The System Bus

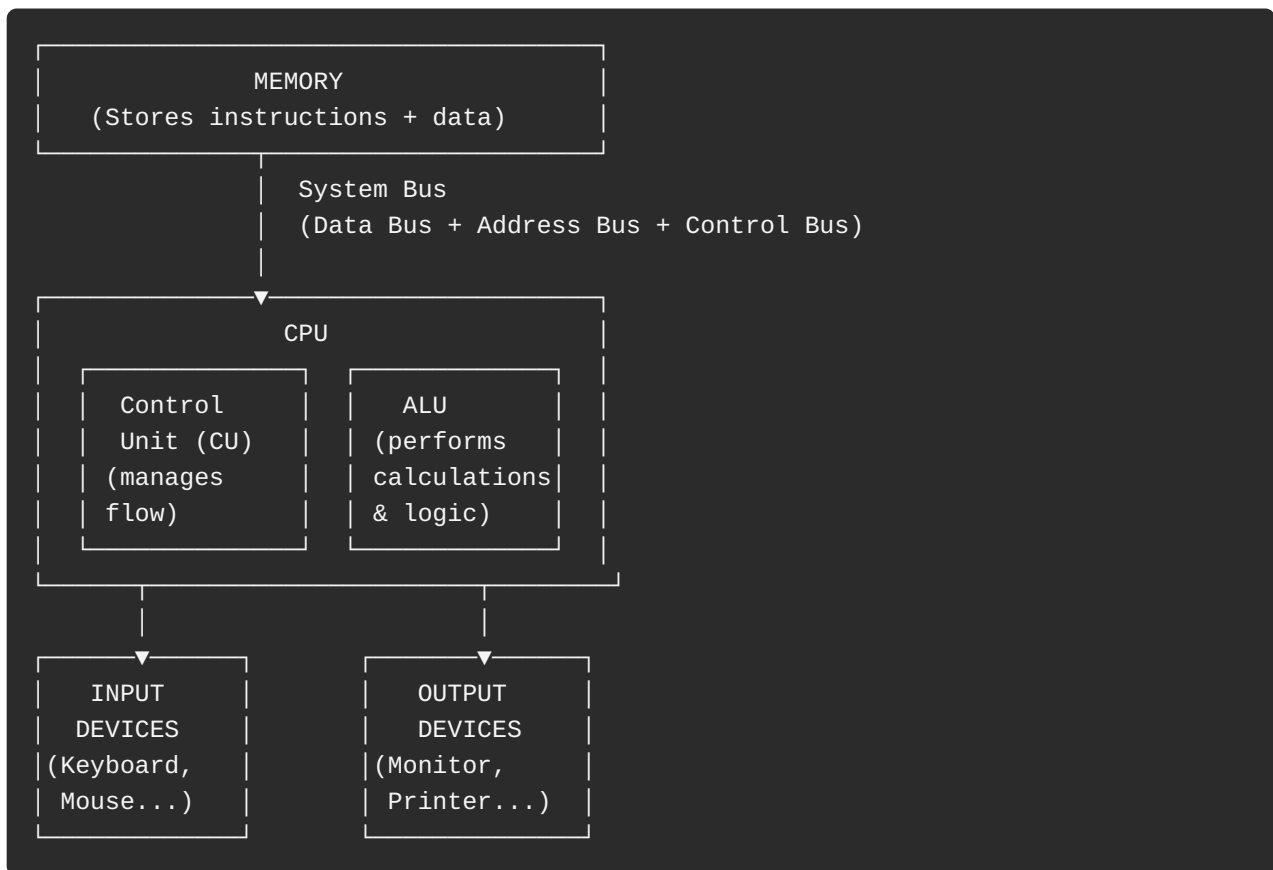
Definition: The System Bus is the communication pathway that moves data between all the components inside the computer.

It has three channels:

Bus	What It Carries
Data Bus	The actual data being transferred.
Address Bus	The memory address — where to read from or write to.
Control Bus	Control signals — timing, read/write instructions.

Think of the system bus as a **highway system** inside the computer. Data, addresses, and control signals all travel on their own dedicated lanes.

The Von Neumann Architecture — Component Map



Working of the Von Neumann Architecture: The Fetch-Decode-Execute-Store Cycle

The CPU executes instructions in a continuous four-step cycle. This cycle repeats millions — sometimes billions — of times per second.

The Hook

Think of a factory assembly line. A worker picks up a part (fetch), reads the instruction sheet (decode), builds it (execute), and places the finished piece in storage (store). Then immediately picks up the next part. Repeat. Forever.

The CPU works exactly like this — a relentless, high-speed assembly line processing instructions.

Step 1: Fetch

What happens: The CPU retrieves the next instruction from memory.

- The **Program Counter (PC)** — a special register inside the CPU — holds the memory address of the next instruction.
- The instruction is retrieved from that address and placed into the **Instruction Register (IR)**.
- The Program Counter automatically updates to point to the **next** instruction.

Key components involved: Memory, Program Counter (PC), Instruction Register (IR).

Step 2: Decode

What happens: The Control Unit reads the instruction in the IR and figures out what it means.

- The instruction contains an **opcode** (operation code) — a binary code that tells the CPU what operation to perform (add, subtract, load data, etc.).
- The Control Unit determines what the CPU needs to do and which data to use.

Key components involved: Control Unit (CU).

Step 3: Execute

What happens: The CPU carries out the instruction.

- If the instruction is a **calculation** (e.g., add two numbers), the ALU performs it.
- If the instruction is a **data movement** (e.g., copy data from one location to another), the Control Unit handles it.

Key components involved: ALU, Control Unit (CU).

Step 4: Store

What happens: The result of the execution is saved — either back to memory or sent to an output device.

- If the result needs to be used later, it goes into **memory (RAM)**.
- If the result is for the user (like text on a screen), it goes to an **output device** (monitor, speakers, printer).

Key components involved: Memory, Output Devices.

The Full Cycle — Example: Adding 2 + 2 in a Calculator App

Step	What Happens	Components Used
Fetch	CPU reads the "ADD" instruction from RAM. PC points to next instruction.	Memory, PC, IR
Decode	Control Unit identifies this as an addition operation. Identifies the two numbers: 2 and 2.	Control Unit
Execute	ALU adds $2 + 2 = 4$.	ALU
Store	Result (4) is sent to the monitor for display.	Memory / Monitor

Interactive Stop-Point: Grab a Partner

Trace through the Fetch-Decode-Execute-Store cycle for this operation:

"The user types the letter 'A' on the keyboard. The letter appears on the screen."

With your partner, answer:

1. What is **fetched** from memory?
2. What does **decoding** determine?
3. What does **execution** involve?
4. Where is the result **stored** or sent?

Quick Recap

The Von Neumann architecture uses a shared memory for instructions and data. The CPU's four-step cycle — Fetch, Decode, Execute, Store — processes every instruction. The ALU does calculations; the Control Unit manages everything.

1.4 Number Systems

Why Do Computers Use Different Number Systems?

Computers work at the hardware level using only two states: **electricity on (1) and electricity off (0)**. Everything a computer stores, processes, or communicates is ultimately represented using these two digits — the **binary** number system.

But binary numbers are long and hard for humans to read. That is why other number systems — octal and hexadecimal — exist: to give us a more compact, human-readable way to represent binary data.

The Hook

When ships communicate at sea, they sometimes use signal flags. Each flag pattern represents a letter or number. The flag system is just a different way of representing the same alphabet — not a different language, just a different representation.

Number systems work the same way. Decimal, binary, octal, and hexadecimal all represent the **same underlying values** — just in different ways, for different purposes.

1.4.1 Decimal Number System (Base 10)

Definition: The decimal system is a **base-10** number system. It uses digits from **0 to 9**. Each digit's value depends on its position (its place value), which is a power of 10.

This is the number system you use every day.

Example: The number 523

Position	Digit	Place Value	Value
Hundreds	5	$10^2 = 100$	$5 \times 100 = 500$
Tens	2	$10^1 = 10$	$2 \times 10 = 20$
Units	3	$10^0 = 1$	$3 \times 1 = 3$
Total			523

So $523 = 5 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 = 500 + 20 + 3 = \mathbf{523}$

1.4.2 Binary Number System (Base 2)

Definition: The binary system is a **base-2** number system. It uses only two digits: **0** and **1**. Each digit's place value is a power of 2.

Binary is the **native language of computers**. Digital circuits have exactly two states — ON (1) and OFF (0). Every piece of data in a computer — text, images, sound, video — is ultimately stored as binary.

Example: The binary number 1011

Position	Digit	Place Value	Value
4th from right	1	$2^3 = 8$	$1 \times 8 = 8$
3rd from right	0	$2^2 = 4$	$0 \times 4 = 0$
2nd from right	1	$2^1 = 2$	$1 \times 2 = 2$
1st from right	1	$2^0 = 1$	$1 \times 1 = 1$
Total			11 in decimal

So $1011_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = \mathbf{11}_{10}$

Converting Decimal to Binary

Algorithm:

1. Divide the decimal number by 2.
2. Record the remainder (0 or 1).
3. Divide the quotient by 2 again.
4. Keep dividing until the quotient reaches 0.
5. Read the remainders **from bottom to top** — that is your binary number.

Example: Convert 83 to binary

Division	Quotient	Remainder
$83 \div 2$	41	1
$41 \div 2$	20	1
$20 \div 2$	10	0
$10 \div 2$	5	0
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1 ← Start reading here

Read remainders **bottom to top**: **1010011**

So $83_{10} = 1010011_2$

Interactive Stop-Point: Pause & Think

Convert these decimal numbers to binary on your own:

1. 15
2. 32
3. 100

(Check your answers: $15 = 1111_2$, $32 = 100000_2$, $100 = 1100100_2$)

1.4.3 Octal Number System (Base 8)

Definition: The octal system is a **base-8** number system. It uses digits from **0 to 7**. Each digit's place value is a power of 8.

Octal was used in early computing systems because it is easy to convert between octal and binary. (The early PDP-8 computer used octal.) Though not widely used in modern computing, it remains important to understand.

Example: The octal number 157

Position	Digit	Place Value	Value
3rd from right	1	$8^2 = 64$	$1 \times 64 = 64$
2nd from right	5	$8^1 = 8$	$5 \times 8 = 40$
1st from right	7	$8^0 = 1$	$7 \times 1 = 7$
Total			111 in decimal

So $157_8 = 1 \times 64 + 5 \times 8 + 7 \times 1 = 111_{10}$

Converting Binary to Octal

Each octal digit represents exactly **3 binary bits** (because $8 = 2^3$).

Method:

1. Group the binary digits into groups of **3 bits**, starting from the right.
2. If the leftmost group has fewer than 3 bits, pad it with zeros on the left.
3. Convert each 3-bit group to its octal digit.

Octal-Binary Correspondence Table:

Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Example: Convert 110101011_2 to octal

```
Group into 3s (from right): 110 | 101 | 011
Convert each group:
  110 = 6
  101 = 5
  011 = 3
```

Result: $110101011_2 = 653_8$

Converting Decimal to Octal

Algorithm:

1. Divide the decimal number by 8.
2. Record the remainder.
3. Divide the quotient by 8 again.
4. Repeat until the quotient is 0.
5. Read remainders **from bottom to top**.

Example: Convert 83 to octal

Division	Quotient	Remainder
$83 \div 8$	10	3
$10 \div 8$	1	2
$1 \div 8$	0	1 ← Start reading here

Read remainders bottom to top: **123**

So $83_{10} = 123_8$

Interactive Stop-Point: Grab a Partner

Work together to:

1. Convert 45_{10} to octal.
2. Convert 128_{10} to octal.
3. Convert 57_8 to decimal.
4. Convert 124_8 to decimal.

Compare your answers with another pair.

1.4.4 Hexadecimal Number System (Base 16)

Definition: The hexadecimal system is a **base-16** number system. It uses digits **0–9** and letters **A–F** (where A = 10, B = 11, C = 12, D = 13, E = 14, F = 15). Each digit's place value is a power of 16.

Hexadecimal (often called "hex") is the most popular shorthand for binary in computing. Memory addresses, color codes in web design (like `#FF5733`), and many system values are written in hexadecimal.

Hexadecimal digit values:

Hex	Decimal
0–9	0–9
A	10
B	11
C	12
D	13
E	14
F	15

Example: The hexadecimal number 1A3

Position	Digit	Place Value	Value
3rd from right	1	$16^2 = 256$	$1 \times 256 = 256$
2nd from right	A (=10)	$16^1 = 16$	$10 \times 16 = 160$
1st from right	3	$16^0 = 1$	$3 \times 1 = 3$
Total			419₁₀

Converting Hexadecimal to Binary

Each hex digit represents exactly **4 binary bits** (because $16 = 2^4$).

Hexadecimal-Binary Correspondence Table:

Hex	Binary	Hex	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

Example: Convert 1101011010110010_2 to hexadecimal

```

Group into 4s (from right): 1101 | 0110 | 1011 | 0010
Convert each group:
  1101 = D
  0110 = 6
  1011 = B
  0010 = 2

```

Result: $1101011010110010_2 = \mathbf{D6B2_{16}}$

Converting Decimal to Hexadecimal

Algorithm:

1. Divide the decimal number by 16.
2. Record the remainder. If the remainder is 10–15, use A–F.
3. Divide the quotient by 16.
4. Repeat until the quotient is 0.
5. Read remainders **from bottom to top**.

Example: Convert 2297 to hexadecimal

Division	Quotient	Remainder	Hex Digit
$2297 \div 16$	143	9	9
$143 \div 16$	8	15	F
$8 \div 16$	0	8	8 ← Start reading here

Read remainders bottom to top: **8F9**

So $2297_{10} = \mathbf{8F9_{16}}$

Interactive Stop-Point: Pause & Think

Web designers write colors in hexadecimal. The color red is written as `#FF0000`.

1. Convert FF_{16} to decimal. What value is that?
 2. What do you think `#000000` represents? And `#FFFFFF`?
 3. Convert the hex number `#1A` to decimal.
-

Quick Recap

Computers work in binary (base 2). Octal (base 8) and hexadecimal (base 16) are compact shorthand for binary. To convert: group binary into 3 bits for octal, 4 bits for hex. To convert decimal: divide repeatedly and read remainders upward.

1.5 Data Representation in Computing Systems

How Computers Store Numbers

Everything stored in a computer is ultimately binary. But there is a question: how many bits do you use? And how do you represent negative numbers?

The Hook

Imagine a set of weighing scales that can only show weights from 0 to 10 kg. If you place 12 kg on it, the scale cannot display the value — it overflows. A computer's memory works similarly: a fixed number of bits can only represent a fixed range of values. Understanding these ranges is essential to writing correct programs.

1.5.1 Whole Numbers (W)

Definition: Whole numbers are non-negative integers: $\{0, 1, 2, 3, \dots\}$

In computing, whole numbers are **unsigned** — they have no sign bit. All bits are used to store the magnitude of the value.

Formula: With n bits, the maximum value that can be stored is $2^n - 1$.

Storage Size	Bits (n)	Maximum Value ($2^n - 1$)
1 byte	8 bits	255
2 bytes	16 bits	65,535
4 bytes	32 bits	4,294,967,295

Example: A 1-byte whole number:

- Maximum: $11111111_2 = 255_{10}$

- Minimum: $00000000_2 = 0_{10}$

1.5.2 Integers (Z) — Signed Numbers

Definition: Integers include both positive and negative numbers: $\{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$

To store negative numbers, one bit is reserved as the **sign bit** — the leftmost (most significant) bit.

Sign Bit Value	Meaning
0	Positive number
1	Negative number

For a 1-byte signed integer (8 bits):

- 1 bit is for the sign.
- 7 bits are for the value.
- Maximum positive value: $01111111_2 = 127_{10}$
- **Formula for maximum:** $2^{(n-1)} - 1$

Storage Size	Bits	Maximum Positive	Minimum Negative
1 byte	8	+127	-128
2 bytes	16	+32,767	-32,768
4 bytes	32	+2,147,483,647	-2,147,483,648

1.5.3 1's Complement

Definition: 1's complement represents a negative binary number by **inverting all the bits** — changing every 0 to 1 and every 1 to 0.

Example: Find the 1's complement of 00000101_2 (which is +5)

```
Original:      0 0 0 0 0 1 0 1
Invert all bits: 1 1 1 1 1 0 1 0
```

1's complement of $00000101_2 = 11111010_2$

1's complement is a stepping stone to 2's complement, which is the actual method computers use.

1.5.4 2's Complement — How Computers Store Negative Numbers

Definition: 2's complement is the standard method computers use to represent negative integers in binary.

Why 2's complement? Because it allows subtraction to be performed using the same addition circuit — only one type of arithmetic circuit is needed in the CPU.

Steps to find the 2's complement:

1. Write the binary of the positive version of the number.
2. **Invert all bits** (find the 1's complement).
3. **Add 1** to the result.

Example: Represent -5 in 8-bit binary using 2's complement

Step 1: Binary of +5:

```
+5 = 000001012
```

Step 2: Invert all bits (1's complement):

```
00000101 → 11111010
```

Step 3: Add 1:

```
  11111010
+         1
-----
  11111011
```

So **-5 in 8-bit 2's complement = 11111011₂**

Minimum Integer Value

For an 8-bit signed integer, the most negative number is represented by:

```
100000002
```

This represents **-128₁₀** (negative 128).

Formula: Minimum value = $-2^{(n-1)}$

Size	Minimum Value
1 byte (8 bits)	$-2^7 = -128$
2 bytes (16 bits)	$-2^{15} = -32,768$
4 bytes (32 bits)	$-2^{31} = -2,147,483,648$

Interactive Stop-Point: Pause & Think

Find the 8-bit 2's complement representation of these negative numbers:

1. -3
2. -10
3. -1

(Hint for -1: Start with 00000001, invert, add 1. What do you get? Is it surprising?)

Quick Recap

Whole numbers are unsigned (all bits = value). Integers are signed (1 bit = sign).
Negative integers are stored using 2's complement: invert all bits, then add 1.

1.6 Binary Arithmetic Operations

Introduction

The CPU performs millions of arithmetic operations every second. All of them are done in binary. Understanding binary arithmetic helps you understand how the CPU actually works.

There are four operations: addition, subtraction, multiplication, and division.

The Hook

Every time you add two numbers on a calculator, play a game, or stream a video — the CPU is performing binary arithmetic. The music coming from your speakers is the result of millions of binary additions per second, processed so fast that it sounds like continuous sound.

1.6.1 Binary Addition

The four rules of binary addition:

Rule	Result	Carry
0 + 0	0	0
0 + 1	1	0
1 + 0	1	0
1 + 1	0	1 (carry 1 to next column)

The special case: $1 + 1 = 10$ in binary (which is 2 in decimal). The "1" carries to the next column, just like in decimal addition when you add $9 + 1 = 10$.

Example: Add $1101_2 + 1011_2$

```

  1 1 0 1
+ 1 0 1 1
-----

```

Work from **right to left**:

Column	Bits	Carry In	Sum	Result Bit	Carry Out
1st (units)	$1 + 1$	0	2	0	1
2nd	$0 + 1$	1	2	0	1
3rd	$1 + 0$	1	2	0	1
4th	$1 + 1$	1	3	1	1
5th (overflow)	—	1	—	1	0

Result: **11000_2**

Let's verify: $1101_2 = 13_{10}$, $1011_2 = 11_{10}$, and $13 + 11 = 24 = 11000_2$. ✓

1.6.2 Binary Subtraction (Using 2's Complement)

Instead of subtracting directly, computers **add the 2's complement** of the number being subtracted. This means the CPU only needs an addition circuit — subtraction is just addition in disguise.

Rule: $A - B = A + (\text{2's complement of } B)$

Example: Subtract 6 from 9 (i.e., 9 - 6)

Step 1: Write both numbers in 4-bit binary:

```
9 = 10012
6 = 01102
```

Step 2: Find the 2's complement of 6:

```
0110 ← original 6
1001 ← invert all bits (1's complement)
+ 1 ← add 1
----
1010 ← 2's complement of 6 (represents -6)
```

Step 3: Add 9 + (-6):

```
 1001
+ 1010
-----
10011
```

Step 4: Discard the carry bit (the extra 1 on the left):

```
10011 → discard carry → 00112 = 310
```

Result: 9 - 6 = 3 ✓

1.6.3 Binary Multiplication

Binary multiplication follows the same **long multiplication** method as decimal, but with simpler rules:

Rule
$0 \times 0 = 0$
$0 \times 1 = 0$
$1 \times 0 = 0$
$1 \times 1 = 1$

Steps:

1. Multiply each bit of the second number (multiplier) by all bits of the first number (multiplicand) — one row per bit.
 2. Shift each row one place to the left compared to the previous row.
 3. Add all the partial products.
-

Example: Multiply $101_2 \times 11_2$

```
  1 0 1   (= 5 in decimal)
×   1 1   (= 3 in decimal)
-----
  1 0 1   (101 × 1 – multiply by rightmost bit, no shift)
 1 0 1 0   (101 × 1 – multiply by next bit, shift left by 1)
-----
 1 1 1 1   (add the two rows)
```

Result: $101_2 \times 11_2 = 1111_2$

Verify: $5 \times 3 = 15 = 1111_2 \checkmark$

1.6.4 Binary Division

Binary division follows the same **long division** method as decimal, but only uses 0s and 1s.

Steps:

1. **Compare:** Compare the divisor with the leftmost portion of the dividend.
 2. **Subtract:** If divisor $\leq$ current portion, subtract it. Write 1 in the quotient.
 3. **Shift:** Bring down the next bit from the dividend.
 4. If the divisor is larger than the current portion, write 0 in the quotient.
 5. **Repeat** until all bits are processed.
-

Example: Divide $1100_2 \div 10_2$

```
Dividend: 11002 (= 12 in decimal)
Divisor:   102 (= 2 in decimal)
Expected:  1102 (= 6 in decimal)
```

Step	Current Portion	Compare with 10_2	Action	Quotient Bit
1	11	$11 \geq 10 \rightarrow$ subtract	$11 - 10 = 01$	1
2	010 (bring down 0)	$010 \geq 10 \rightarrow$ subtract	$010 - 010 = 000$	1
3	000 (bring down 0)	$000 < 10$	No subtraction	0

Result: $110_2 = 6 \checkmark$

Class Activity: Practicing Binary Division

Form groups of 3–4 students. Solve these binary division problems. Show every step.

- $10101_2 \div 10_2$
- $11100_2 \div 11_2$
- $100110_2 \div 101_2$

Present your solutions to the class. Verify by converting to decimal first.

Interactive Stop-Point: Grab a Partner

Perform these binary operations. Show your working.

- $1010_2 + 0110_2$
- $1111_2 - 0101_2$ (use 2's complement)
- $110_2 \times 101_2$

Verify each answer by converting to decimal and checking.

Quick Recap

Binary addition uses 4 rules; $1+1=10$ with a carry. Subtraction uses 2's complement addition. Multiplication uses long multiplication. Division uses long division. All four follow the same logic as decimal arithmetic — just in base 2.

1.7 Common Text Encoding Schemes

How Computers Represent Text

Computers only understand numbers (in binary). So how does a computer store the letter "A" or the word "Pakistan"?

The answer: every character is assigned a number. The agreed-upon system of which number maps to which character is called a **text encoding scheme**.

The Hook

Imagine two spies communicating in secret. They agree on a code: A = 1, B = 2, C = 3, and so on. When one spy writes "8-5-12-12-15", the other reads "HELLO."

This is exactly how text encoding works. The computer and the software agree on a code: A = 65, B = 66, and so on. When the computer stores "A", it stores the number 65 in binary.

1.7.1 ASCII

Definition: ASCII stands for **American Standard Code for Information Interchange**. It is the original standard encoding system for computers, assigning a number (0–127) to every standard character.

ASCII assigns a unique number from **0 to 127** to every:

- Letter (uppercase and lowercase)
- Digit (0–9)
- Punctuation mark (., !, ?, etc.)
- Special character (space, tab, etc.)

Key ASCII values to know:

Character	ASCII Code
Space	32
A (uppercase)	65
B (uppercase)	66
Z (uppercase)	90
a (lowercase)	97
b (lowercase)	98
z (lowercase)	122
0 (digit)	48
9 (digit)	57

Example: Encoding "Pakistan" in ASCII

Letter	ASCII Code	Binary
P	80	01010000
a	97	01100001
k	107	01101011
i	105	01101001
s	115	01110011
t	116	01110100
a	97	01100001
n	110	01101110

So the word "Pakistan" is stored in a computer as the sequence of binary numbers above.

Standard ASCII: 7 bits, 128 characters (0–127).

Extended ASCII: 8 bits, 256 characters (0–255). Adds accented letters, extra symbols.

Class Activity

1. Write your full name.
2. Find the ASCII code for each letter (use the ASCII table).

3. Convert each ASCII code to binary.
 4. You have just written your name in binary — the same way a computer stores it!
-

1.7.2 Unicode

The problem with ASCII: ASCII was designed for English. It can represent 128 characters — enough for English letters, numbers, and punctuation. But what about Urdu? Arabic? Chinese? Hindi? Japanese? Each of these languages has hundreds or thousands of characters.

ASCII cannot represent them. A new standard was needed.

Definition: Unicode is a universal character encoding standard that assigns a unique code to every character in every language in the world. It can represent over **1 million characters**.

Unicode uses the notation **U+** followed by a hexadecimal code for each character.

- **A** = U+0041
- **ب** (Urdu letter Ba) = U+0628

Unicode has multiple encoding forms — UTF-8, UTF-16, and UTF-32 — which differ in how many bytes they use per character.

UTF-8

Definition: UTF-8 is a **variable-length** encoding. It uses between 1 and 4 bytes per character.

Key property: UTF-8 is **backward compatible with ASCII**. Any text file written in ASCII can be read by a UTF-8 system without any changes. This made UTF-8 easy to adopt — existing ASCII content worked without modification.

How many bytes?

- Standard English characters (like A, B, C): **1 byte** (same as ASCII).
- Extended characters (accented letters, common symbols): **2 bytes**.
- Characters from most languages (Arabic, Urdu, Hindi, Chinese): **2–3 bytes**.
- Rare or ancient characters: **4 bytes**.

Examples:

Character	Unicode	UTF-8 Binary	Bytes
A	U+0041	01000001	1 byte
ب (Urdu Ba)	U+0628	11011000 10101000	2 bytes

UTF-16

Definition: UTF-16 is a **variable-length** encoding that uses either **2 or 4 bytes** per character.

Unlike UTF-8, UTF-16 is **not** compatible with ASCII. It is commonly used internally by operating systems like Windows and programming languages like Java.

Examples:

Character	UTF-16 Binary	Bytes
A	00000000 01000001	2 bytes
ب (Urdu Ba)	00000110 00101000	2 bytes

UTF-32

Definition: UTF-32 is a **fixed-length** encoding that always uses exactly **4 bytes** per character.

Advantage: Very simple. Every character takes the same space.

Disadvantage: Uses more memory — even for simple English text, every character uses 4 bytes instead of 1.

Example:

Character	UTF-32 Binary	Bytes
A	00000000 00000000 00000000 01000001	4 bytes

Comparison: ASCII vs. Unicode Encodings

Feature	ASCII	UTF-8	UTF-16	UTF-32
Characters	128	1,000,000+	1,000,000+	1,000,000+
Bytes per character	1	1–4	2–4	4 (fixed)
ASCII compatible?	—	Yes	No	No
Best for	English only	Web, most uses	Windows, Java	Simplicity

1.7.3 Data Storage Units

All files — text, images, audio, video — are stored as binary data on storage devices. Data size is measured in bytes and its multiples.

Unit	Value
1 Bit	A single binary digit: 0 or 1
1 Byte (B)	8 bits
1 Kilobyte (KB)	1,024 bytes
1 Megabyte (MB)	1,024 KB
1 Gigabyte (GB)	1,024 MB
1 Terabyte (TB)	1,024 GB
1 Petabyte (PB)	1,024 TB
1 Exabyte (EB)	1,024 PB
1 Zettabyte (ZB)	1,024 EB
1 Yottabyte (YB)	1,024 ZB

Common file sizes for reference:

- A single character in ASCII: **1 byte**
- A typical text document (5 pages): **~25 KB**
- An MP3 song: **~4–5 MB**
- A full HD movie: **~4–8 GB**
- A large hard drive: **1–4 TB**

Interactive Stop-Point: Pause & Think

A school wants to digitize all its text records. They estimate they have 10,000 pages of text. A typical page is about 2,000 characters.

1. How many characters total? ($10,000 \times 2,000 = ?$)
 2. In ASCII, each character = 1 byte. How many bytes total?
 3. Convert that to KB, then MB.
 4. Could you fit all these records on a USB drive with 1 GB of storage?
-

Quick Recap

ASCII maps 128 characters to numbers (0–127) — enough for English. Unicode extends this to over 1 million characters for all world languages. UTF-8, UTF-16, and UTF-32 are different ways to encode Unicode. All data is stored as binary — measured in bytes and their multiples.

Chapter Summary

This chapter introduced you to the foundational concepts of computational systems — how computers are designed, how they process information, and how they store and represent all types of data.

Topic	Key Takeaway
System Theory	A system is interdependent parts with a shared goal. Every system has: objective, components, environment, and communication.
System Software	Manages hardware. Examples: OS, device drivers, utilities.
Application Software	Helps users do tasks. Examples: Word, Chrome, games.
Von Neumann Architecture	Shared memory for data and instructions. Components: Memory, CPU (ALU + CU), Input, Output, System Bus.
Fetch-Decode-Execute-Store	The four-step cycle the CPU repeats to execute every instruction.
Decimal (Base 10)	Digits 0–9. Daily life number system. Place values are powers of 10.
Binary (Base 2)	Digits 0–1. Computer's native language. Place values are powers of 2.
Octal (Base 8)	Digits 0–7. Each octal digit = 3 binary bits.
Hexadecimal (Base 16)	Digits 0–9 and A–F. Each hex digit = 4 binary bits. Compact binary shorthand.
Whole Numbers	Unsigned (all bits = value). Max value = $2^n - 1$.
Integers	Signed (1 bit = sign). Max = $2^{(n-1)} - 1$. Min = $-2^{(n-1)}$.
1's Complement	Invert all bits. A stepping stone to 2's complement.
2's Complement	Invert all bits, then add 1. Standard method for storing negative numbers.
Binary Addition	$1+1=10$ (carry). Four simple rules.
Binary Subtraction	Add the 2's complement. Discard carry.
Binary Multiplication	Long multiplication with shifts.
Binary Division	Long division — compare, subtract, shift, repeat.
ASCII	128 characters, 7 bits. English letters, digits, symbols.
Unicode (UTF-8/16/32)	1M+ characters. Covers all world languages. UTF-8 most common.
Data Storage Units	Bit → Byte → KB → MB → GB → TB and beyond.

Key Vocabulary

Term	Definition
System	An organized collection of interdependent parts working toward a common goal.
Objective	The goal or purpose a system is designed to achieve.
Environment	Everything external to a system that interacts with it.
Software	Programs and instructions that tell a computer what to do.
System Software	Software that manages hardware and provides a platform for applications.
Application Software	Software that helps users perform specific tasks.
Von Neumann Architecture	A computer design with shared memory for both programs and data.
CPU	Central Processing Unit — the brain of the computer.
ALU	Arithmetic Logic Unit — performs calculations and logical operations.
Control Unit (CU)	Manages the flow of instructions through the CPU.
System Bus	The communication pathway connecting all computer components.
Fetch-Decode-Execute-Store	The four-step cycle the CPU uses to process every instruction.
Binary	Base-2 number system using only 0 and 1.
Decimal	Base-10 number system (0–9) used in everyday life.
Octal	Base-8 number system (0–7). Each digit = 3 binary bits.
Hexadecimal	Base-16 number system (0–9, A–F). Each digit = 4 binary bits.
Unsigned Number	A binary number with no sign bit — only positive values.
Signed Number	A binary number with a sign bit — can store positive and negative values.
1's Complement	Inverting all bits of a binary number.
2's Complement	Inverting all bits then adding 1 — the standard for storing negative numbers.
ASCII	American Standard Code for Information Interchange. 128-character encoding.

Term	Definition
Unicode	Universal character encoding covering 1M+ characters from all languages.
UTF-8	Variable-length Unicode encoding (1–4 bytes). Backward compatible with ASCII.
UTF-16	Variable-length Unicode encoding (2–4 bytes).
UTF-32	Fixed-length Unicode encoding (4 bytes per character).
Bit	A single binary digit: 0 or 1.
Byte	8 bits. The basic unit of data storage.

Review Questions

Multiple Choice Questions

- The primary function of a system is:
 - a) To work independently
 - b) **To achieve a common goal ✓**
 - c) To create new systems
 - d) To provide entertainment
- One of the fundamental concepts of any system is:
 - a) Its size
 - b) **Its objective ✓**
 - c) Its age
 - d) Its price
- An example of a simple system is:
 - a) A human body
 - b) A computer network
 - c) **A thermostat regulating temperature ✓**
 - d) The Internet
- The basic concepts used to describe a system are:
 - a) Users, hardware, software
 - b) **Objectives, components, environment, communication ✓**
 - c) Inputs, outputs, processes
 - d) Sensors, actuators, controllers

5. Which describes the Von Neumann architecture's main characteristic?
- a) Separate memory for data and instructions
 - b) Parallel execution of instructions
 - c) **Single memory store for both program instructions and data** ✓
 - d) Multiple CPUs for different tasks
6. ASCII stands for:
- a) **American Standard Code for Information Interchange** ✓
 - b) Advanced Standard Code for Information Interchange
 - c) American Standard Communication for Information Interchange
 - d) Advanced Standard Communication for Information Interchange
7. How many bits are used in standard ASCII encoding?
- a) **7 bits** ✓
 - b) 8 bits
 - c) 16 bits
 - d) 32 bits
8. How many bytes are typically used to store an integer?
- a) 1 byte
 - b) 2 bytes
 - c) **4 bytes** ✓
 - d) 8 bytes
9. Which software is used to enhance system performance and security?
- a) Operating system
 - b) **Utility software** ✓
 - c) Application software
 - d) Device drivers
10. Which of the following is an example of application software?
- a) **Microsoft Word** ✓
 - b) BIOS
 - c) Disk Cleanup
 - d) Device Manager
-

Short Questions

1. Define a system.
2. List the main components of the Von Neumann architecture.
3. Name the four main steps in the Von Neumann instruction cycle.
4. What is a key advantage of the Von Neumann architecture?

5. What is the primary purpose of ASCII?
 6. Define Unicode. How is it different from ASCII?
 7. How does the number of bits affect the range of integer values that can be stored?
 8. Define system software and provide two examples.
 9. Differentiate between system software and application software. Give two examples of each.
 10. Add 1100_2 and 1011_2 . Show your working.
 11. Subtract 0011_2 from 1010_2 using 2's complement. Show all steps.
-

Long Questions

1. Describe the basic concept of a system. Explain its four fundamental concepts with examples.
 2. Explain the Von Neumann architecture. Name and describe all its key components.
 3. Explain the working of the Von Neumann architecture using the Fetch-Decode-Execute-Store cycle. Use a real-world example.
 4. Explain how characters are encoded using Unicode. Discuss UTF-8, UTF-16, and UTF-32 with examples, including an Urdu character.
 5. Describe in detail how integers (both positive and negative) are stored in computer memory. Explain the role of 2's complement with a worked example.
 6. Perform the following binary arithmetic operations and verify your answers in decimal:
 - a. Multiply 101_2 by 11_2
 - b. Divide 1100_2 by 10_2
-

Practical Exercises

Exercise 1 — Number System Conversion Table

Complete this table without a calculator:

Decimal	Binary	Octal	Hexadecimal
10	?	?	?
25	?	?	?
83	?	?	?
255	?	?	?
16	?	?	?

Exercise 2 — Binary Arithmetic Practice

Solve each of the following. Show all working. Verify in decimal.

1. $10110_2 + 01101_2$
2. $11010_2 - 01011_2$ (use 2's complement)
3. $1011_2 \times 101_2$
4. $11010_2 \div 10_2$

Exercise 3 — 2's Complement

Find the 8-bit 2's complement representation of:

1. -7
2. -15
3. -64
4. -128

Exercise 4 — ASCII Activity

1. Look up the ASCII table.
2. Encode your full name in ASCII decimal codes.
3. Convert each decimal code to binary.
4. Now decode this ASCII message: 72 - 101 - 108 - 108 - 111

Exercise 5 — Storage Size Calculation

A digital library wants to store 5,000 books. Each book has an average of 300 pages, and each page has 2,500 characters.

1. How many total characters?
2. In UTF-8 (assuming all English), how many bytes?
3. Convert to MB and then to GB.
4. How many USB drives (each 32 GB) would be needed?

End of Unit 1: Introduction to Computational Systems

You have just looked inside the machine. You now know why computers use binary, how the CPU processes instructions one tiny step at a time, how your name is stored as a string of 0s and 1s, and how negative numbers can be represented with clever bit tricks. This is the foundation of everything in computer science. Every program ever written runs on these principles. You have the foundation. Now build on it.